



**POLITÉCNICO
ESTELLA**

**ASIGNATURA: PROGRAMACIÓN DE SERVICIOS Y
PROCESOS**

**GRADO SUPERIOR EN DESARROLLO DE
APLICACIONES MULTIPLATAFORMA**

PSP04 TAREA 4

Presentado por: Eugen Moga

Fecha: 14/02/2026

ÍNDICE

Ejercicio 1.....	1
Ejercicio 2.....	9

Ejercicio 1

En el **servidor** que har  de banco declaro la variable PUERTO para indicar el puerto y la variable INTERES que la utilizare para calcular el inter s.

```
public class Servidor {  
  
    private static final int PUERTO = 8000;  
  
    private static final double INTERES = 0.02;
```

Dentro del m todo principal (main) declaro e inicializo a nulo de momento el ServerSocket para el servidor y el socket para el cliente

```
public static void main(String[] args) {  
  
    // ServerSocket que escucha conexiones entrantes  
    ServerSocket servidorSocket = null;  
  
    // Socket representa la conexion con un cliente  
    Socket clienteSocket = null;
```

Dentro de un bloque try-catch para poder tratar las excepciones le indico que el servidorSocket escucha en el puerto 8000 y muestro en pantalla los mensajes que estoy esperando conexi n

```
try {  
    // El servidor escucha en el puerto 8000  
    servidorSocket = new ServerSocket(PUERTO);  
    System.out.println("\n    BANCO    ");  
    System.out.println("Esperando conexion en el puerto: " + PUERTO + "\n");
```

Le indico que el servidor acepta la conexión del cliente y muestro por pantalla el mensaje: Cliente conectado

```
// Se acepta la conexion
clienteSocket = servidorSocket.accept();
System.out.println("Cliente conectado");
```

Creo el flujo de entrada y de salida con DataInputStream y DataOutputStream respectivamente para poder recibir y enviar datos con el cliente

```
// Se crea los flujos de entrada y salida
DataInputStream flujo_entrada = new DataInputStream(clienteSocket.getInputStream());
DataOutputStream flujo_salida = new DataOutputStream(clienteSocket.getOutputStream());
```

Declaro la variable capital e indico que flujo de entrada lea la entrada del usuario.

```
// Se recibe el capital a invertir del cliente
double capital = flujo_entrada.readDouble();
System.out.println("Capital a invertir: " + capital);
```

Declaro la variable ganancia y le indico que la ganancia es igual a capital por interés y envío la respuesta al cliente mediante el flujo de salida

```
// Se calcula la ganancia
double ganancia = capital * INTERES;

// Se envia la gancia al cliente
flujo_salida.writeDouble(ganancia);
System.out.println("Mensaje enviado al cliente.");
```

Cierro las conexiones de flujo de entrada, flujo de salida y el Socket del cliente

```
// Cierro las conexiones
flujo_entrada.close();
flujo_salida.close();
clienteSocket.close();
System.out.println("Conexion cerrada");
```

Dentro del bloque catch trato las excepciones de entrada salida

```
} catch (IOException e) {
    System.out.println("Error en el servidor: " + e.getMessage());
    e.printStackTrace();
}
```

Y en el bloque finally cierro el Socket del servidor

```
}finally{
    // Se cierra el servidor al finalizar
    try {
        if (servidorSocket != null && !servidorSocket.isClosed()) {
            servidorSocket.close();
            System.out.println("Servidor cerrado correctamente ");
        }
    } catch (IOException e) {
        System.out.println("Error al cerrar el servidor: " + e.getMessage());
    }
}
```

Creo la clase **Ciente** que hará de cliente del banco, declaro las variables HOST y PUERTO que las voy a utilizar para establecer la conexión en el Socket

```
public class Ciente {  
  
    static final String HOST = "localhost";  
    static final int PUERTO = 8000;
```

Dentro del método principal (main) declaro la variable socket de tipo Socket y la inicializo a nulo. Y declaro la variable sc de tipo Scanner para leer la entrada del usuario.

```
public static void main(String[] args) {  
  
    // Representa la conexion con el servidor  
    Socket socket = null;  
  
    // Scanner para leer los datos del usuario por teclado  
    Scanner sc = new Scanner(System.in);
```

Dentro del bloque try-catch muestro el mensaje CLIENTE y le indico al socket el host y el puerto donde se tiene que conectar.

```
try {  
    System.out.println("\n    CLIENTE    \n");  
  
    // Conectar con el servidor  
    socket = new Socket(HOST, PUERTO);  
    System.out.println("Conectado con el banco");
```

Creo el flujo de entrada y de salida con `DataInputStream` y `DataOutputStream` respectivamente para poder recibir y enviar datos con el servidor

```
// Se crean los flujos de entrada salida
DataInputStream flujo_entrada = new DataInputStream(socket.getInputStream());
DataOutputStream flujo_salida = new DataOutputStream(socket.getOutputStream());
```

Solicito que el usuario ingrese la cantidad y lo asigno a la variable `capitalInvertido`. Con el bloque `if` valido que el número sea mayor que 0. Con `flujo_salida` le envío la respuesta al servidor.

```
double capitalInvertido;

while (true) {
    System.out.print("Ingresa el capital a invertir: ");
    capitalInvertido = sc.nextDouble();

    // Valido que el capital sea positivo
    if (capitalInvertido > 0) {
        break;
    } else{
        System.out.println("El capital debe ser mayor que cero");
    }
}
```

Con `flujo_salida` le envio la cantidad al servidor

```
// Se envia el capital al servidor
flujo_salida.writeDouble(capitalInvertido);
```

Con `flujo_entrada` leo la respuesta del servidor y la asigno a la variable `ganancia`.

```
// Recibo la respuesta
double ganancia = flujo_entrada.readDouble();
```

Muestro los mensajes de información, como capital invertido, ganancia y el total de capital mas ganancia, formateado mostrando solo 2 decimales

```
System.out.println("\n RESULTADO \n");  
System.out.printf("Capital invertido: €%.2f\n", capitalInvertido);  
System.out.printf("Ganancia mensual: €%.2f\n", ganancia);  
System.out.printf("Total despues de 1 mes: €%.2f\n", (capitalInvertido+ganancia));
```

Cierro las conexiones

```
// Se cierra la conexion  
flujo_entrada.close();  
flujo_salida.close();  
socket.close();  
sc.close();  
System.out.println("Conexion cerrada");
```

Dentro del bloque catch trato las excepciones y finalmente cierro el socket

```
} catch (UnknownHostException e) {  
    System.out.println("No se puede encontrar el servidor: " + e.getMessage());  
} catch (IOException e){  
    System.out.println("Error de comunicacion: " + e.getMessage());  
} finally{  
    try {  
        if(socket != null && !socket.isClosed()){  
            socket.close();  
        }  
    } catch (IOException e) {  
        System.out.println("Error al cerrar la conexion: " + e.getMessage());  
    }  
}  
}
```

Para probarlo el correcto funcionamiento ejecuto el servidor y veo que me muestra el mensaje BANCO y Esperando conexión

```
Output - Servidor (run)

run:
Picked up JAVA_TOOL_OPTIONS: -Dfile.encoding=UTF-8

BANCO
Esperando conexion en el puerto: 8000
```

Ahora ejecuto el cliente y me muestra el mensaje Conectado con el banco e Ingresar el capital a invertir:

```
Output

Servidor (run) x Cliente (run) x

run:
Picked up JAVA_TOOL_OPTIONS: -Dfile.encoding=UTF-8 -Ds

CLIENTE

Conectado con el banco
Ingresa el capital a invertir:
```

En este momento si voy al servidor me muestra el mensaje Cliente conectado

```
Output

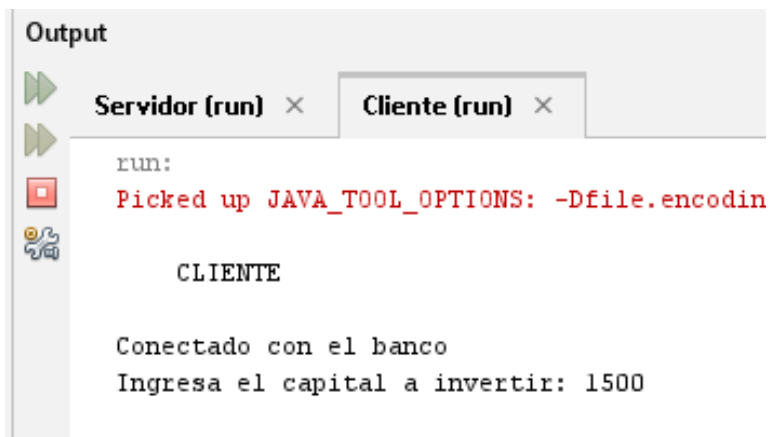
Servidor (run) x Cliente (run) x

run:
Picked up JAVA_TOOL_OPTIONS: -Dfile.encoding

BANCO
Esperando conexion en el puerto: 8000

Cliente conectado
```

Vuelvo al Cliente y le ingreso la cantidad a invertir:

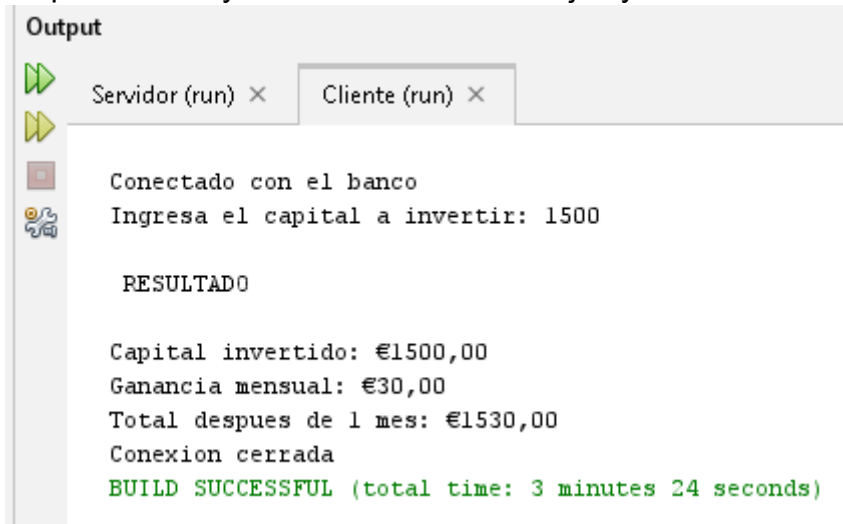


The screenshot shows the IDE's Output window with two tabs: 'Servidor (run)' and 'Cliente (run)'. The 'Cliente (run)' tab is active. The output text is as follows:

```
run:
Picked up JAVA_TOOL_OPTIONS: -Dfile.encoding=
CLIENTE

Conectado con el banco
Ingresa el capital a invertir: 1500
```

Le pulso Enter y me muestra los mensajes y cierra la conexión



The screenshot shows the IDE's Output window with the 'Cliente (run)' tab active. The output text is as follows:

```
Conectado con el banco
Ingresa el capital a invertir: 1500

RESULTADO

Capital invertido: €1500,00
Ganancia mensual: €30,00
Total despues de 1 mes: €1530,00
Conexion cerrada
BUILD SUCCESSFUL (total time: 3 minutes 24 seconds)
```

Ejercicio 2

Para el ejercicio 2 he reutilizado las clases del ejercicio 1 tanto para el servidor como para el cliente (no voy a explicar otra vez el mismo código) explicare lo que he cambiado.

En la clase servidor he creado la variable DESCUENTO para calcular el descuento.

```
public class ServidorTienda {  
  
    private static final int PUERTO = 8000;  
  
    private static final double DESCUENTO = 0.25;  
}
```



Y he modificado los mensajes y los nombres de las variables para que sean más intuitivo

Para la clase ClienteTienda he declarado una variable para calcular el precio total

```
public class ClienteTienda {  
  
    static final String HOST = "localhost";  
    static final int PUERTO = 8000;  
    private static double precioTotal;  
}
```



Con un bucle for solicito al usuario que ingrese 2 precios para 2 productos y con el bucle while valido que no se salga del bucle si el usuario no ingresa un nuemro correcto

```
// Solicito dos precios al usuario
for(int i = 0; i < 2; i++){
    double precio;
    while (true) {
        System.out.print("Ingresa el precio del producto " + (i+1) + ": ");
        precio = sc.nextDouble();

        if (precio > 0) {
            break;
        }else{
            System.out.println("El precio debe ser mayor que cero");
        }
    }
    precioTotal += precio;
}
```

Envio el precio total al servidor y me devuelve el descuento. Muestro los mensajes personalizados

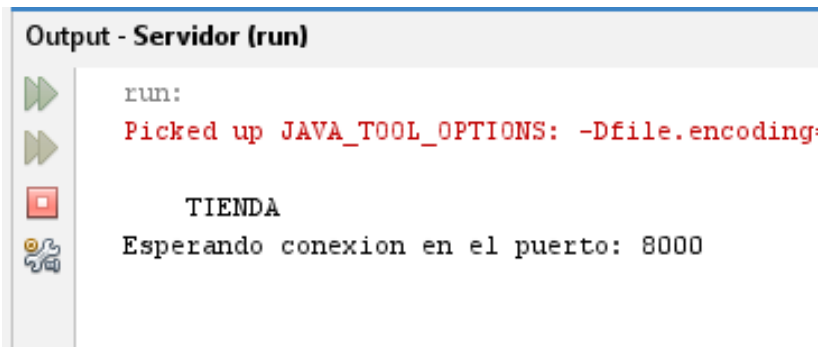
```
// Se envia el capital al servidor
flujo_salida.writeDouble(precioTotal);

// Recibo la respuesta
double descuento = flujo_entrada.readDouble();

System.out.println("\n DESCUENTO");
System.out.printf("Precio sin descuento: €%.2f\n", precioTotal);
System.out.printf("Descuento: €%.2f\n", descuento);
System.out.printf("Total a pagar: €%.2f\n", (precioTotal-descuento));
```

Paso a ejecutar el Servidor Tienda para probar el correcto funcionamiento.

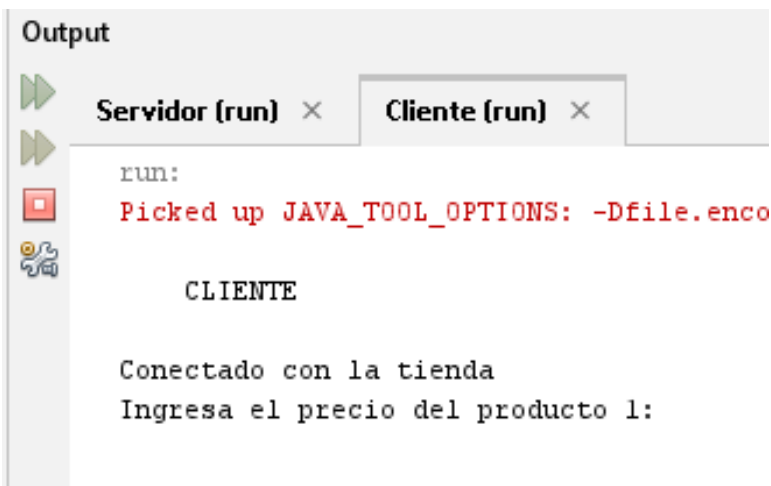
Me muestra el mensaje TIENDA Esperando conexión...,



```
run:
Picked up JAVA_TOOL_OPTIONS: -Dfile.encoding:

TIENDA
Esperando conexion en el puerto: 8000
```

Ejecuto Cliente Tienda me muestra el mensaje TIENDA y Conectado con la tienda. Seguido solicita ingresar el precio del producto 1

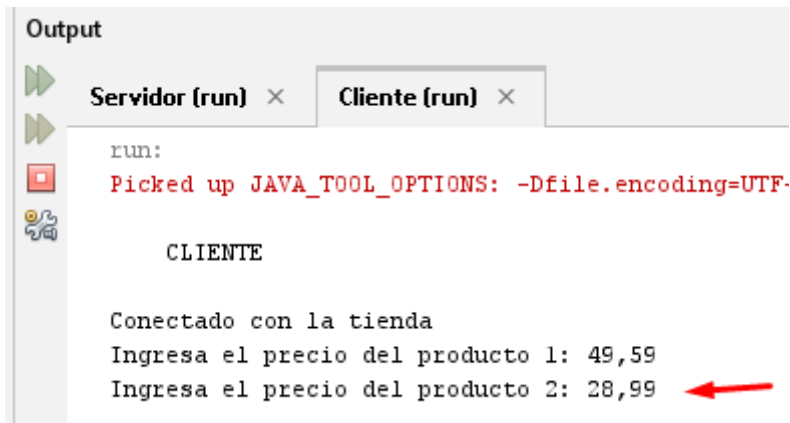


```
run:
Picked up JAVA_TOOL_OPTIONS: -Dfile.enco

CLIENTE

Conectado con la tienda
Ingresa el precio del producto 1:
```

Ingreso el precio del producto 1 y le pulso Enter. Y solicita el precio del segundo producto

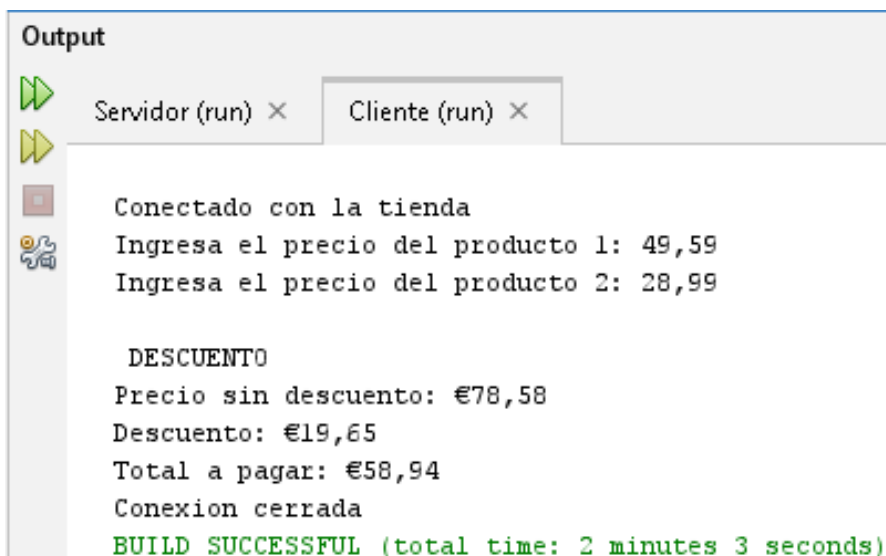


```
run:
Picked up JAVA_TOOL_OPTIONS: -Dfile.encoding=UTF-8

CLIENTE

Conectado con la tienda
Ingresa el precio del producto 1: 49,59
Ingresa el precio del producto 2: 28,99
```

Al pulsar Enter la segunda vez el programa envía el precio total al servidor. El servidor calcula el 25% del total y lo devuelve al cliente. En el cliente muestro por pantalla el precio sin descuento, el descuento y total a pagar



```
Conectado con la tienda
Ingresa el precio del producto 1: 49,59
Ingresa el precio del producto 2: 28,99

DESCUENTO
Precio sin descuento: €78,58
Descuento: €19,65
Total a pagar: €58,94
Conexion cerrada
BUILD SUCCESSFUL (total time: 2 minutes 3 seconds)
```